

DevOps avancé
Année universitaire 2026–2027

Travaux pratiques, TP1

Continuous Deployment avec GitOps et ArgoCD

Un dépôt Git décrit l'état désiré du cluster. ArgoCD le surveille, l'applique, corrige toute dérive et permet de revenir à une version stable.

Réalisé par

Kacem Mathlouthi
Oussema Kraiem
Med Amin Haouas

Dépôt

`github.com/KacemMathlouthi/tp-devops`

Image

`ghcr.io/kacemmathlouthi/tp-devops`

Table des matières

Présentation générale 3
- Objectif 3
- Application 3
- Construction de l'image 3
- Stack technique 4
- Architecture 4

Étape 1 : structure du dépôt Git 5
- Organisation 5
- Base Kustomize 5
- Overlay `dev` 5

Étape 2 : installation et configuration d'ArgoCD 7
- Cluster Kubernetes 7
- Installation d'ArgoCD 7
- Accès à l'interface 8
- Création de l'Application 8

Démo 1 : déploiement continu automatisé 12
- Préparation : image v2 12
- Changement dans Git 12
- Synchronisation par ArgoCD 12

Démo 2 : drift et self-healing 15
- Suppression du Deployment 15
- Drift sur un champ : réplicas 16

Démo 3 : Sync Status et Health Status 19
- Deux indicateurs indépendants 19
- Erreur introduite dans Git 19
- Synced, puis Degraded 19

Démo 4 : historique et rollback 22
- Désactivation de l'auto-sync 22
- Historique 22
- Rollback 23
- Retour à Git 24

Conclusion 26
- Concepts démontrés 26
- Limites et pistes 26

Présentation générale

Objectif

- Décrire l'application et son infrastructure Kubernetes de façon déclarative dans Git.
- Laisser ArgoCD, installé dans le cluster, appliquer cet état : le cluster tire sa configuration, aucun `kubectl apply` externe.
- Démontrer cinq concepts :

Single Source of Truth	tout l'état désiré vit dans Git
Pull-based CD	chaque commit est appliqué automatiquement par ArgoCD
Drift et self-healing	toute modification manuelle du cluster est annulée
Sync et Health	conforme à Git (Synced) ne veut pas dire fonctionnel (Healthy)
Rollback et historique	retour instantané à une révision stable

Application

- API `FastAPI` (Python 3.12, `uv`), libre choix laissé par le sujet.
- `GET /` : page affichant la version déployée et le pod qui répond.
- `GET /version` : même information en JSON.
- `GET /healthz` : sonde readiness / liveness.
- Version injectée au build (`ARG APP_VERSION`) : chaque déploiement est visible dans le navigateur.

Construction de l'image

- GitHub Action (`.github/workflows/build.yml`) déclenchée par un tag Git `v*`.
- Build puis push vers `ghcr.io/kacemmathlouthi/tp-devops:`.
- La CI n'a aucun accès au cluster : elle produit l'image, ArgoCD la déploie.

```
git tag v2 && git push origin v2   # → ghcr.io/kacemmathlouthi/tp-devops:v2
```

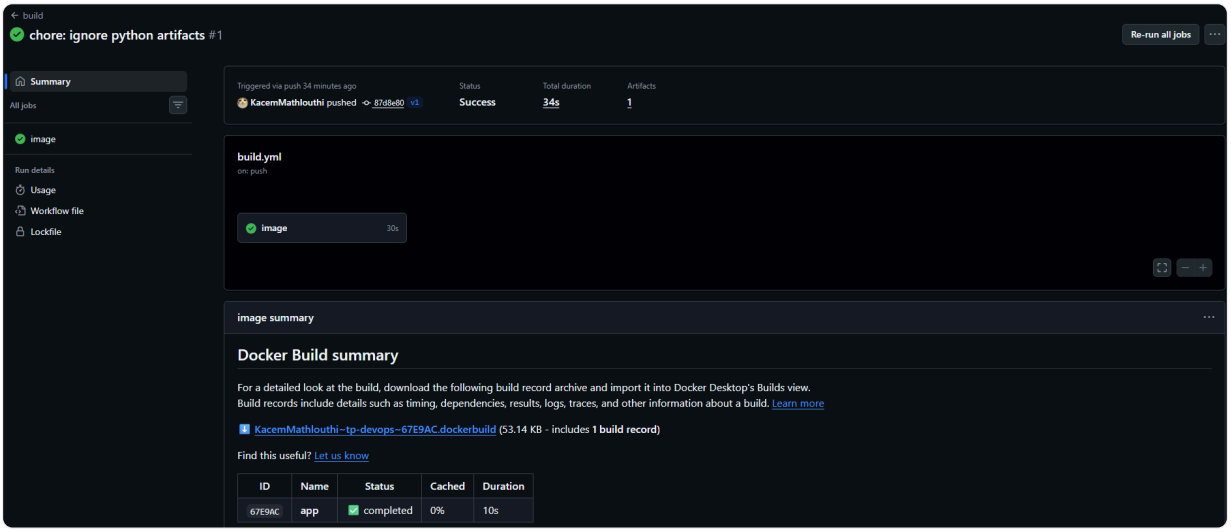


Fig. 1. Le push du tag `v1` déclenche le workflow `build` (34 s)

Stack technique

Cluster	minikube (driver Docker) sous WSL 2
GitOps	ArgoCD v3.5.5, namespace `argocd`
Manifests	Kustomize : `base` + overlay `dev`
Dépôt	GitHub public `KacemMathlouthi/tp-devops`, branche `main`
Application	FastAPI, image Docker non-root
CI	GitHub Actions → GHCR

Architecture

```
Développeur          GitHub (SSOT)                    Cluster minikube
──────────           ──────────────                   ────────────────
git push ──────────► tp-devops (main)      ◄──pull──  argocd
git tag vX            ├── app/                         ├── repo-server     : rend Kustomize
                      └── k8s/overlays/dev             ├── app-controller  : compare & sync
                             │                         └── server (UI)     : localhost:8080
                             ▼ tag vX                          │ apply
                      GitHub Actions                           ▼
                             │ docker push              tp-gitops
                             ▼                          ├── Deployment web
                      ghcr.io/ .../tp-devops:vX ◄pull── ├── Pods (FastAPI)
                                                        └── Service web
```

Étape 1 : structure du dépôt Git

Organisation

- Un seul dépôt : code, CI, manifests, Application ArgoCD et rapport.
- ArgoCD ne lit que `k8s/overlays/dev` ; le reste n'affecte pas la synchronisation.

```
tp-devops/
├── app/                          # code applicatif + Dockerfile
├── .github/workflows/build.yml   # build + push de l'image sur tag vX
├── k8s/
│   ├── base/                     # Deployment, Service, kustomization
│   └── overlays/dev/             # namespace, réplicas, tag d'image
├── argocd/application.yaml       # l'Application ArgoCD
└── compte-rendu/                 # ce rapport
```

Base Kustomize

- Décrit l'application indépendamment de l'environnement : `Deployment` + `Service` + labels communs.
- `readinessProbe` / `livenessProbe` sur `/healthz` : base du Health Status d'ArgoCD.
- `POD_NAME` injecté via l'API downward.
- Requests / limits CPU et mémoire.
- `progressDeadlineSeconds: 60` : un rollout bloqué passe en échec en 1 min au lieu de 10.
- Service `ClusterIP` 80 → 8000 sur le label `app: web`.

```
# k8s/base/kustomization.yaml
labels:
  - pairs:
      app.kubernetes.io/name: web
      app.kubernetes.io/part-of: tp-gitops
resources:
  - deployment.yaml
  - service.yaml
```

Overlay `dev`

- Seul fichier modifié pendant les démonstrations.
- Fixe le namespace, le nombre de réplicas et le tag de l'image.
- Rendu vérifiable sans cluster : `kubectl kustomize k8s/overlays/dev`.

```
# k8s/overlays/dev/kustomization.yaml
namespace: tp-gitops
resources:
  - ../../base
replicas:
  - name: web
    count: 2
images:
```

```
  - name: ghcr.io/kacemmathlouthi/tp-devops
    newTag: v1
```

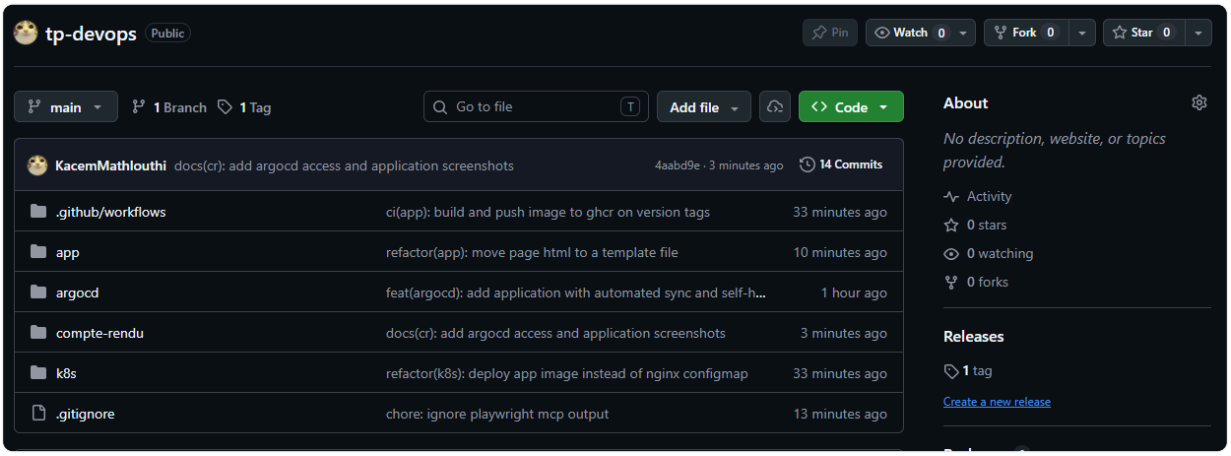


Fig. 2. Dépôt public `KacemMathlouthi/tp-devops`, avec le package GHCR lié

Étape 2 : installation et configuration d'ArgoCD

Cluster Kubernetes

- minikube mono-nœud, driver Docker, sous WSL 2.
- 2 vCPU et 4 Go de RAM suffisent pour Kubernetes, ArgoCD et l'application.

```
minikube start --driver=docker --cpus=2 --memory=4g
kubectl config use-context minikube
```

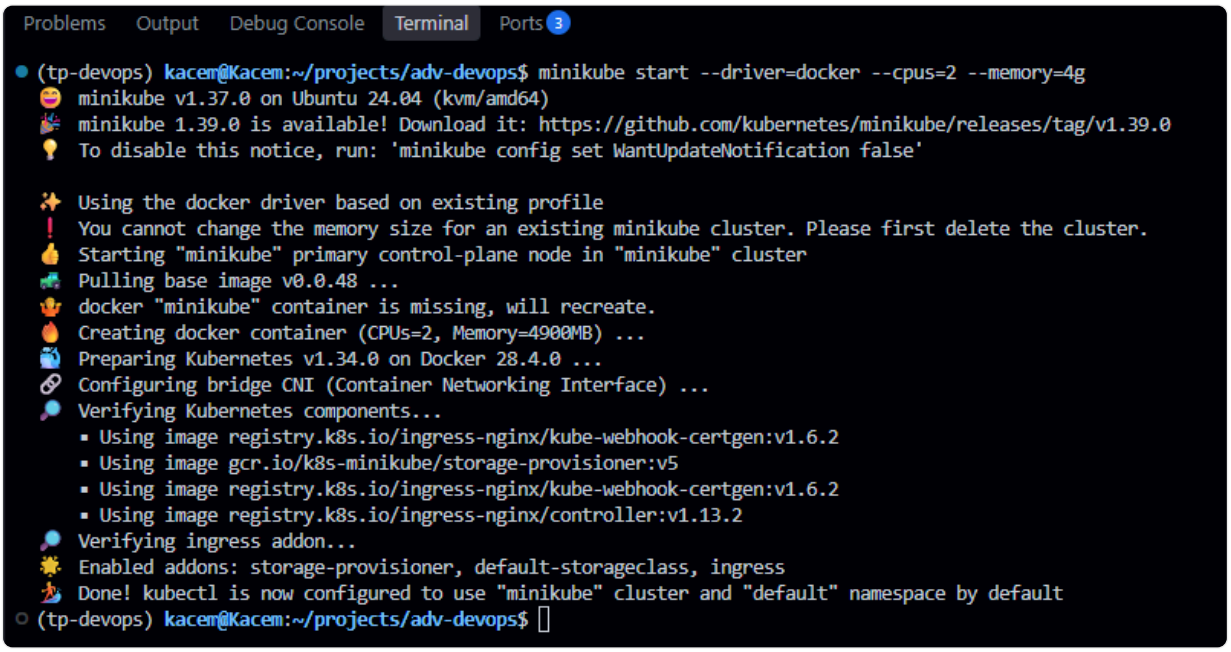


Fig. 3. Démarrage du cluster (Kubernetes v1.34.0)

Installation d'ArgoCD

- Manifests officiels dans le namespace `argocd`.
- `-server-side` requis : certaines CRD dépassent la taille de l'annotation `last-applied-configuration`.

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
```

```
Problems  Output  Debug Console  Terminal  Ports 3

(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n argocd rollout status deploy/argocd-server
Waiting for deployment "argocd-server" rollout to finish: 0 of 1 updated replicas are available...
deployment "argocd-server" successfully rolled out
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n argocd get pods
NAME                                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0                     1/1     Running   0          5m40s
argocd-applicationset-controller-7c7cd47f9d-m26kk   1/1     Running   0          5m41s
argocd-dex-server-6698896998-zm767                  1/1     Running   0          5m41s
argocd-notifications-controller-644457c445-fq9r8    1/1     Running   0          5m41s
argocd-redis-6bcd6966f-n24t4                        1/1     Running   0          5m41s
argocd-repo-server-6f6657d5cb-nqdkx                 1/1     Running   0          5m41s
argocd-server-664495b675-qlw9n                      1/1     Running   0          5m41s
(tp-devops) kacem@Kacem:~/projects/adv-devops$
```

Fig. 4. Les 7 pods ArgoCD en état `Running`

Accès à l'interface

```
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8080:443
```

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d; echo
<redacted>
(tp-devops) kacem@Kacem:~/projects/adv-devops$
```

Fig. 5. Mot de passe initial de `admin` (masqué)

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n argocd port-forward svc/argocd-server 8080:443
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Handling connection for 8080
Handling connection for 8080
```

Fig. 6. UI exposée sur `https://localhost:8080`

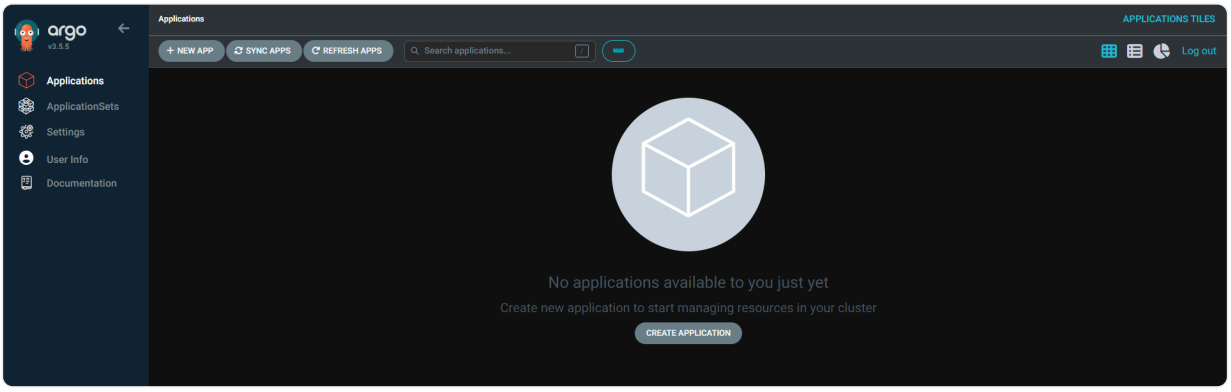


Fig. 7. Première connexion : aucune application

Création de l'Application

- Décrite dans Git (`argocd/application.yaml`), comme le reste.
- Seule commande `kubectl apply` du TP : ArgoCD crée ensuite lui-même le namespace et les ressources.

```
spec:
  source:
    repoURL: https://github.com/KacemMathlouthi/tp-devops.git
    targetRevision: main
    path: k8s/overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: tp-gitops
  syncPolicy:
    automated:
      prune: true      # supprime ce qui a disparu de Git
      selfHeal: true   # annule les modifications manuelles
    syncOptions:
      - CreateNamespace=true
```

automated	synchronise dès qu'un nouveau commit est détecté
selfHeal	réapplique Git si le cluster dérive
prune	supprime du cluster ce qui est retiré de Git
Polling	Git interrogé toutes les 3 min ; Refresh force une vérification

```
kubectl apply -f argocd/application.yaml
```

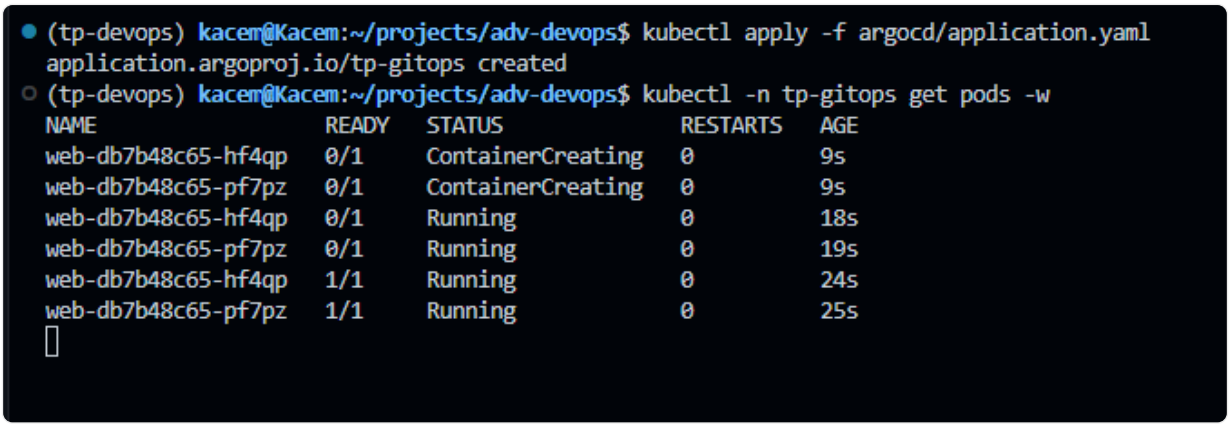


Fig. 8. Création de l'Application : les deux pods passent `Running 1/1` en moins de 30 s

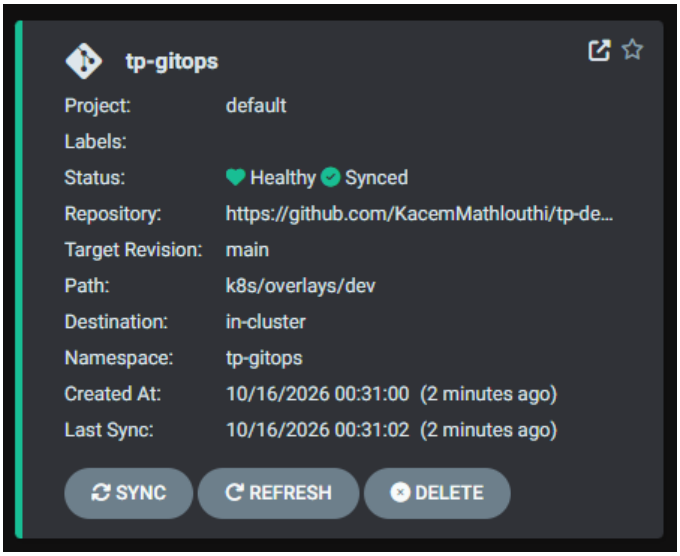


Fig. 9. Application `tp-gitops` : Healthy et Synced

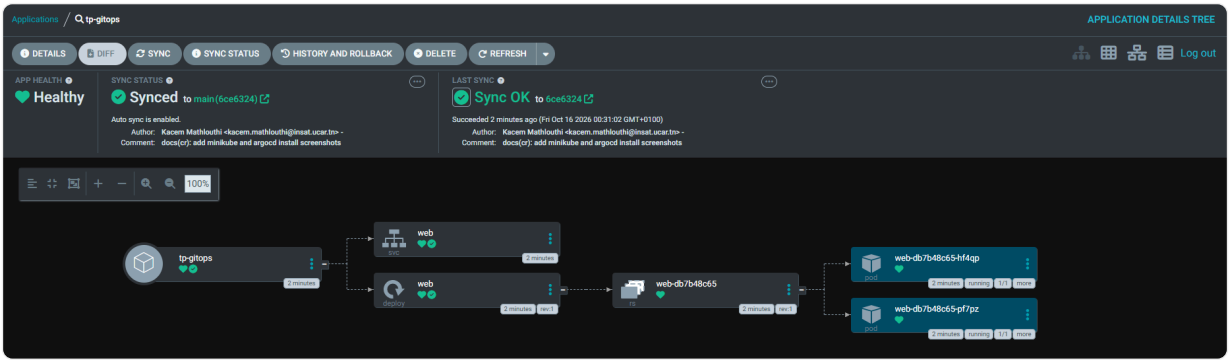


Fig. 10. Arbre des ressources : Service, Deployment, ReplicaSet, 2 pods

– App Health et Sync Status sont affichés séparément : deux indicateurs indépendants (démo 3).

```
kubectl -n tp-gitops port-forward svc/web 8081:80
```



Fig. 11. Application servie par le cluster : version `v1`

Démo 1

Démo 1 : déploiement continu automatisé

Objectif : un commit suffit à modifier le cluster, sans commande `kubectl`.

Préparation : image v2

- Nouvelle version de la page, publiée par un tag Git.
- La CI produit l'image mais ne déploie rien : tant que Git ne la référence pas, le cluster ne change pas.

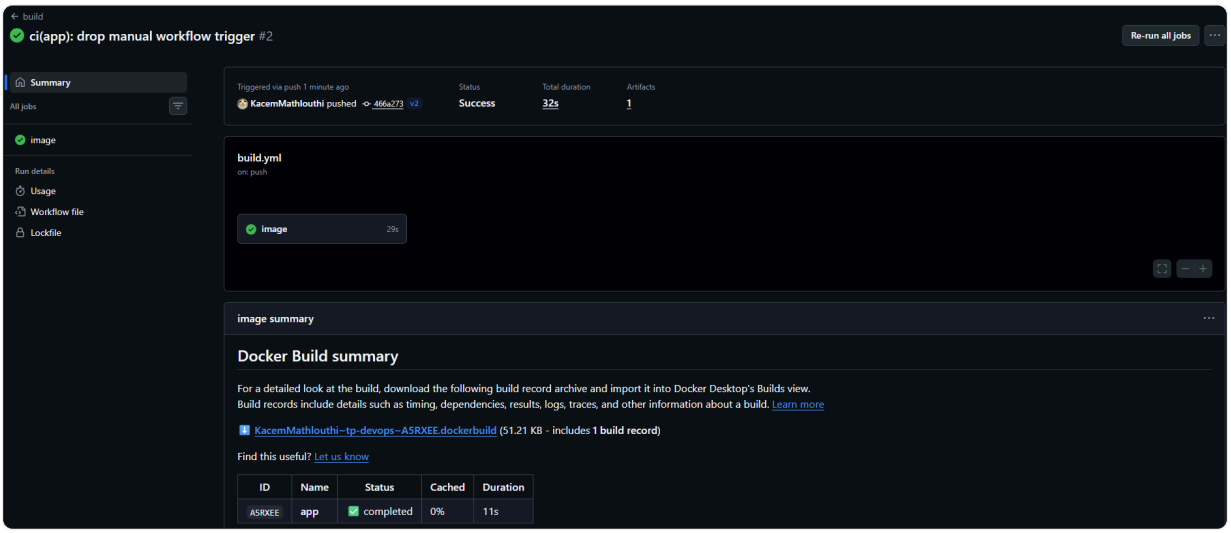


Fig. 12. Le tag `v2` construit et publie `tp-devops:v2` (32 s)

Changement dans Git

```
# k8s/overlays/dev/kustomization.yaml
-    newTag: v1
+    newTag: v2
```

```
git commit -am "feat(k8s): deploy v2" && git push
```

Synchronisation par ArgoCD

- Nouveau commit détecté (polling ou Refresh), appliqué par la politique `automated`.
- Kubernetes effectue un rolling update (`maxSurge` / `maxUnavailable` à 25 %) :
  - un pod v2 démarre ;
  - un pod v1 n'est supprimé qu'une fois le pod v2 prêt ;
  - jamais moins de deux pods disponibles.

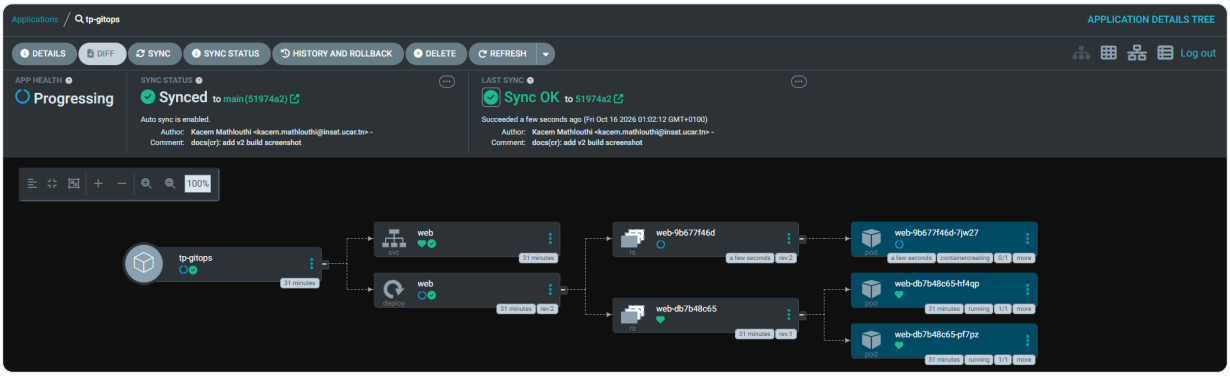


Fig. 13. Synced mais Progressing : le ReplicaSet v2 démarre son premier pod, les pods v1 servent encore

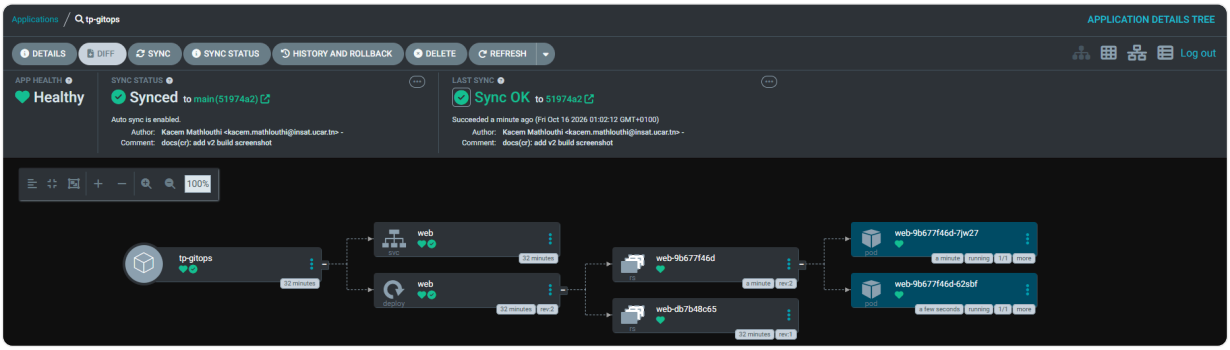


Fig. 14. Healthy et Synced : 2 pods v2, l'ancien ReplicaSet conservé à 0

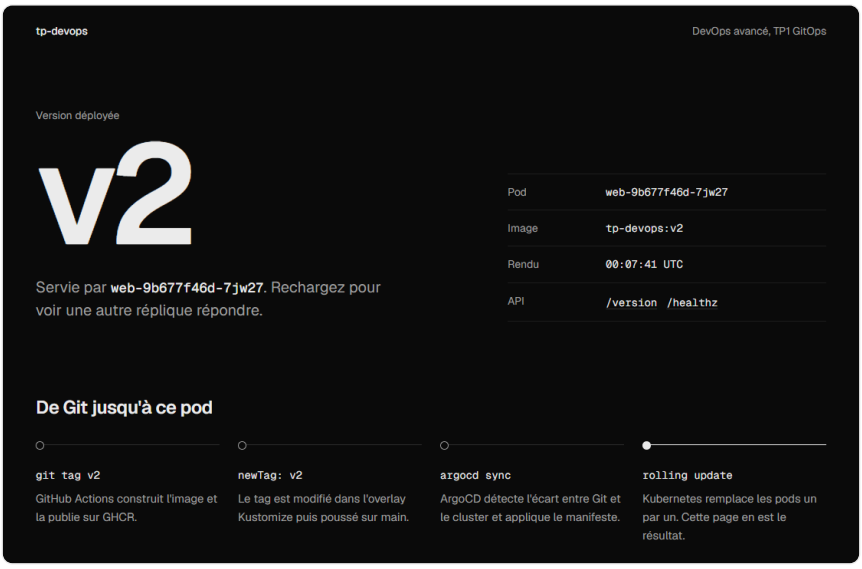


Fig. 15. La page affiche la version `v2`

- **Résultat**
  - Un commit d'une ligne a déployé la v2.
  - Aucune intervention sur le cluster, aucune interruption de service.

Démo 2

Démo 2 : drift et self-healing

Objectif : modifier le cluster à la main et voir ArgoCD rétablir l'état de Git.

Suppression du Deployment

```
kubectl -n tp-gitops delete deployment web
```

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n tp-gitops delete deployment web
deployment.apps "web" deleted from tp-gitops namespace
(tp-devops) kacem@Kacem:~/projects/adv-devops$
```

Fig. 16. Suppression manuelle du Deployment, sans passer par Git

- L'application-controller surveille les ressources gérées via l'API Kubernetes (watches).
- L'écart est détecté immédiatement, sans attendre le polling de Git.
- `selfHeal: true` relance aussitôt une synchronisation.

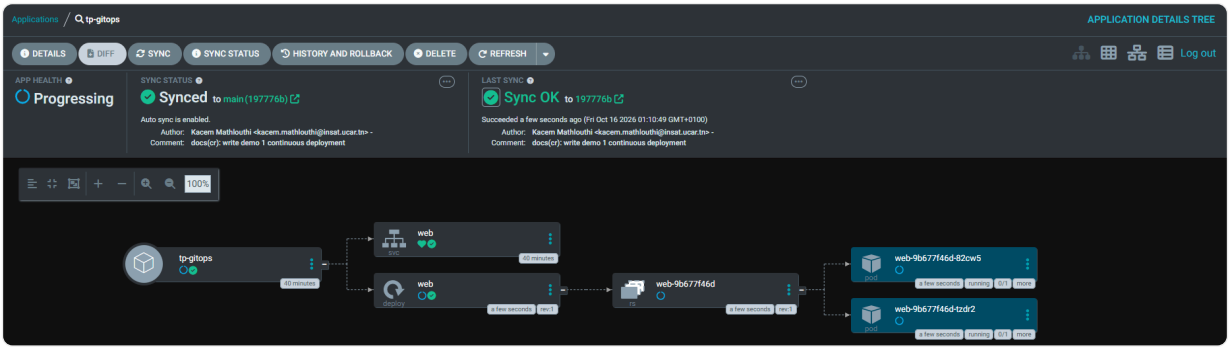


Fig. 17. Quelques secondes après : Deployment déjà recréé, pods en démarrage (Progressing)

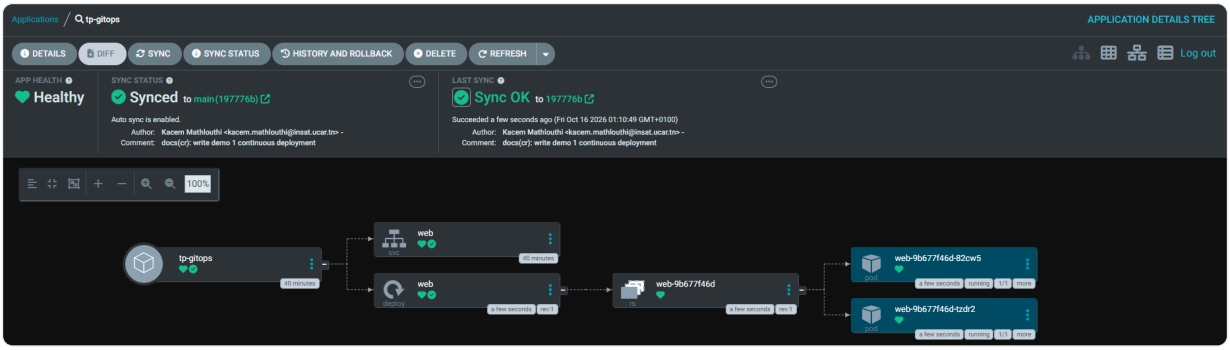


Fig. 18. 15 s après : Healthy et Synced

- Preuves de la recréation :
  - le Service garde son âge (40 min), le Deployment et les pods sont neufs ;
  - le Deployment repart en `rev:1` : son historique a disparu avec lui ;
  - nouveaux noms de pods, même hash de template (même manifeste Git).

Drift sur un champ : réplicas

```
kubectl -n tp-gitops scale deploy web --replicas=5
```

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n tp-gitops scale deploy web --replicas=5
deployment.apps/web scaled
(tp-devops) kacem@Kacem:~/projects/adv-devops$
```

Fig. 19. Passage manuel à 5 réplicas



Fig. 20. ArgoCD rétablit `replicas: 2` : les pods en trop sont déjà en `terminating`

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n tp-gitops get pods -w
NAME                  READY   STATUS              RESTARTS   AGE
web-9b677f46d-82cw5   1/1     Running             0          8m44s
web-9b677f46d-tzdr2   1/1     Running             0          8m44s
web-9b677f46d-ckrfd   0/1     Pending             0          0s
web-9b677f46d-jprv5   0/1     Pending             0          0s
web-9b677f46d-zrkrf   0/1     Pending             0          0s
web-9b677f46d-ckrfd   0/1     Pending             0          0s
web-9b677f46d-jprv5   0/1     Pending             0          0s
web-9b677f46d-zrkrf   0/1     Pending             0          0s
web-9b677f46d-ckrfd   0/1     ContainerCreating   0          0s
web-9b677f46d-zrkrf   0/1     ContainerCreating   0          0s
web-9b677f46d-jprv5   0/1     ContainerCreating   0          0s
web-9b677f46d-ckrfd   0/1     Terminating         0          0s
web-9b677f46d-zrkrf   0/1     Terminating         0          0s
web-9b677f46d-jprv5   0/1     Terminating         0          0s
web-9b677f46d-zrkrf   0/1     Terminating         0          2s
web-9b677f46d-jprv5   0/1     Terminating         0          2s
web-9b677f46d-ckrfd   0/1     Terminating         0          2s
web-9b677f46d-zrkrf   1/1     Terminating         0          7s
web-9b677f46d-jprv5   1/1     Terminating         0          7s
web-9b677f46d-ckrfd   1/1     Terminating         0          7s
web-9b677f46d-zrkrf   0/1     Error               0          35s
web-9b677f46d-jprv5   0/1     Error               0          35s
web-9b677f46d-ckrfd   0/1     Error               0          35s
web-9b677f46d-zrkrf   0/1     Error               0          36s
web-9b677f46d-zrkrf   0/1     Error               0          36s
web-9b677f46d-jprv5   0/1     Error               0          36s
web-9b677f46d-jprv5   0/1     Error               0          36s
web-9b677f46d-jprv5   0/1     Error               0          36s
web-9b677f46d-ckrfd   0/1     Error               0          36s
web-9b677f46d-ckrfd   0/1     Error               0          36s
web-9b677f46d-ckrfd   0/1     Error               0          36s
```

Fig. 21. Les 3 pods supplémentaires passent en `Terminating` dès leur création

– Correction si rapide que l'état OutOfSync n'est même pas visible.

- **Résultat**

  – Toute modification faite hors de Git est annulée en quelques secondes.
  – Pour changer durablement le cluster, il faut passer par un commit.

Démo 3

Démo 3 : Sync Status et Health Status

Objectif : une application peut être synchronisée avec Git tout en étant défaillante.

Deux indicateurs indépendants

Sync Status	manifests du cluster = manifests de Git ? Synced / OutOfSync
Health Status	les ressources fonctionnent-elles ? Healthy / Progressing / Degraded

Erreur introduite dans Git

```
# k8s/overlays/dev/kustomization.yaml
-     newTag: v2
+     newTag: doesnotexist
```

```
git commit -am "feat(k8s): deploy nonexistent image" && git push
```

Synced, puis Degraded

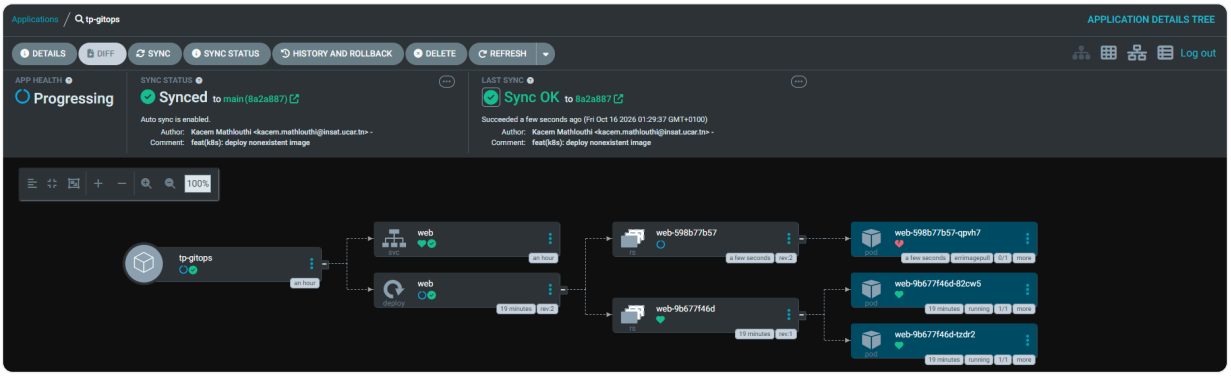


Fig. 22. Synced mais Progressing : le nouveau pod échoue en `errimagepull`

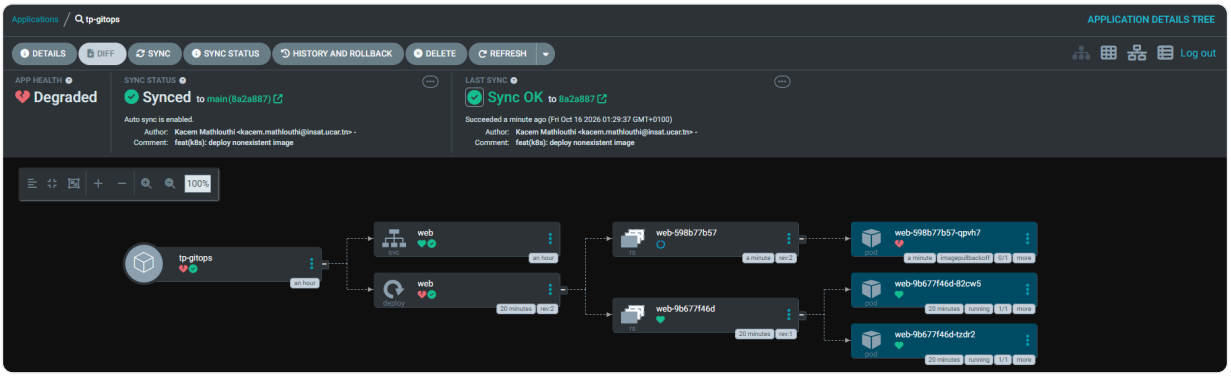


Fig. 23. Après 60 s : toujours Synced, mais Degraded

– Le passage à Degraded vient de Kubernetes : `ProgressDeadlineExceeded` après `progressDeadlineSeconds` (60 s).

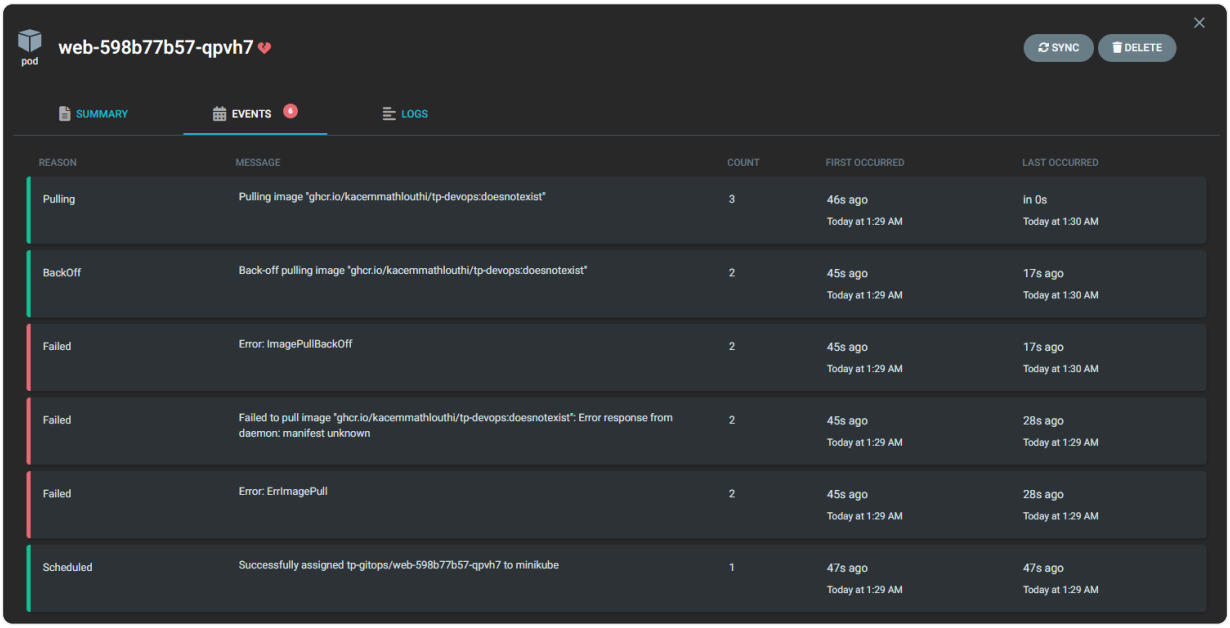


Fig. 24. Événements du pod : `manifest unknown`, puis `ErrImagePull` / `ImagePullBackOff`

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n tp-gitops get pods -w
NAME                  READY   STATUS              RESTARTS   AGE
web-9b677f46d-82cw5   1/1     Running             0          18m
web-9b677f46d-tzdr2   1/1     Running             0          18m
web-598b77b57-qpvh7   0/1     Pending             0          0s
web-598b77b57-qpvh7   0/1     Pending             0          0s
web-598b77b57-qpvh7   0/1     ContainerCreating   0          0s
web-598b77b57-qpvh7   0/1     ErrImagePull        0          2s
web-598b77b57-qpvh7   0/1     ImagePullBackOff    0          18s
web-598b77b57-qpvh7   0/1     ErrImagePull        0          30s
web-598b77b57-qpvh7   0/1     ImagePullBackOff    0          47s
web-598b77b57-qpvh7   0/1     ErrImagePull        0          62s
web-598b77b57-qpvh7   0/1     ImagePullBackOff    0          77s
web-598b77b57-qpvh7   0/1     ErrImagePull        0          2m2s
web-598b77b57-qpvh7   0/1     ImagePullBackOff    0          2m18s
```

Fig. 25. Nouvelles tentatives à intervalles croissants ; les deux pods v2 restent `Running`

- `ErrImagePull` : la tentative de pull vient d'échouer.
- `ImagePullBackOff` : attente avant la tentative suivante (délai croissant).
- Pas d'interruption : le pod cassé n'est jamais prêt, donc les pods v2 ne sont pas supprimés.

Résultat

- Synced : ArgoCD a appliqué exactement ce que dit Git.
- Degraded : ce que dit Git ne fonctionne pas.
- La synchronisation garantit la conformité à Git, pas le bon fonctionnement.

Démo 4

Démo 4 : historique et rollback

Objectif : revenir à une version stable depuis History and Rollback.

Point de départ : Synced mais Degraded sur `8a2a887` (image `doesnotexist`).

Désactivation de l'auto-sync

- ArgoCD refuse le rollback tant que l'auto-sync est actif.
- Sinon, il réappliquerait aussitôt la révision de Git, donc la version cassée.

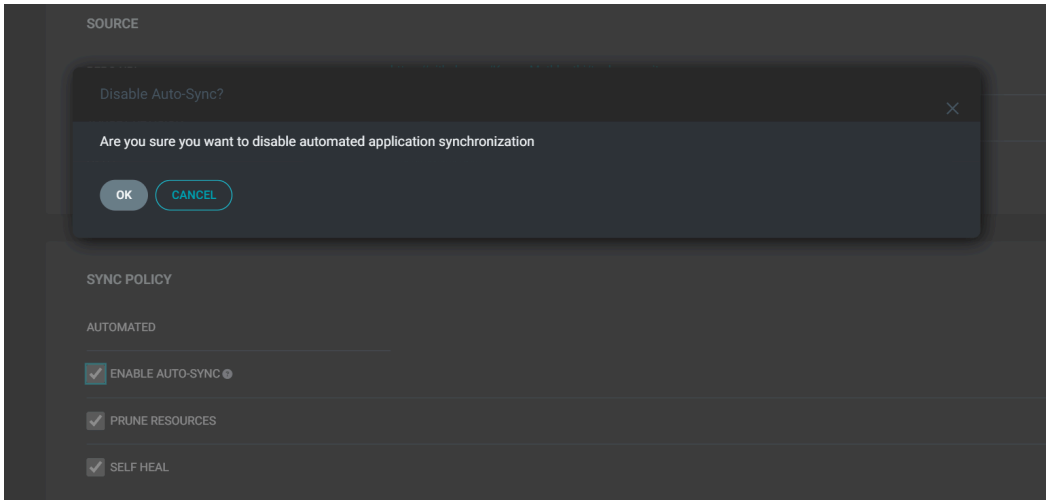


Fig. 26. Désactivation de la synchronisation automatique

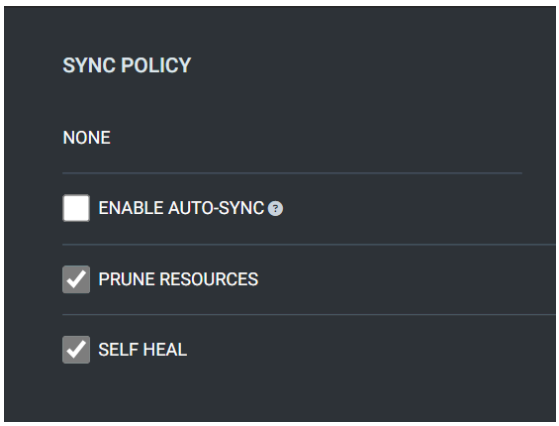


Fig. 27. Sync Policy à None

Historique

- Chaque synchronisation est enregistrée : révision, auteur, message, date, origine.
- 10 entrées conservées (`revisionHistoryLimit`).
- La v2 apparaît sous un message `docs(cr)` : ArgoCD enregistre la tête de branche au moment de la synchronisation.

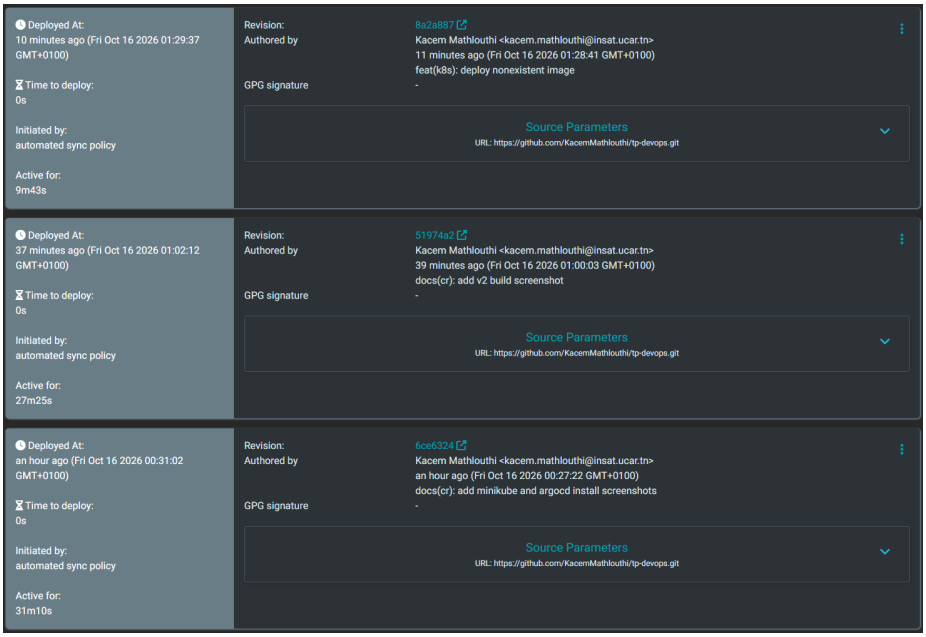


Fig. 28. `8a2a887` (cassée), `51974a2` (v2), `6ce6324` (v1)

Rollback

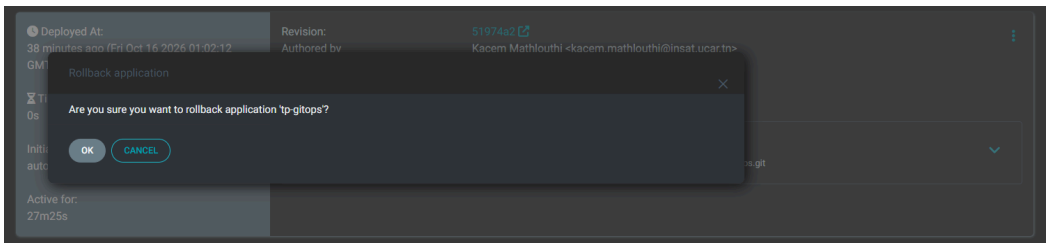


Fig. 29. Rollback vers `51974a2`

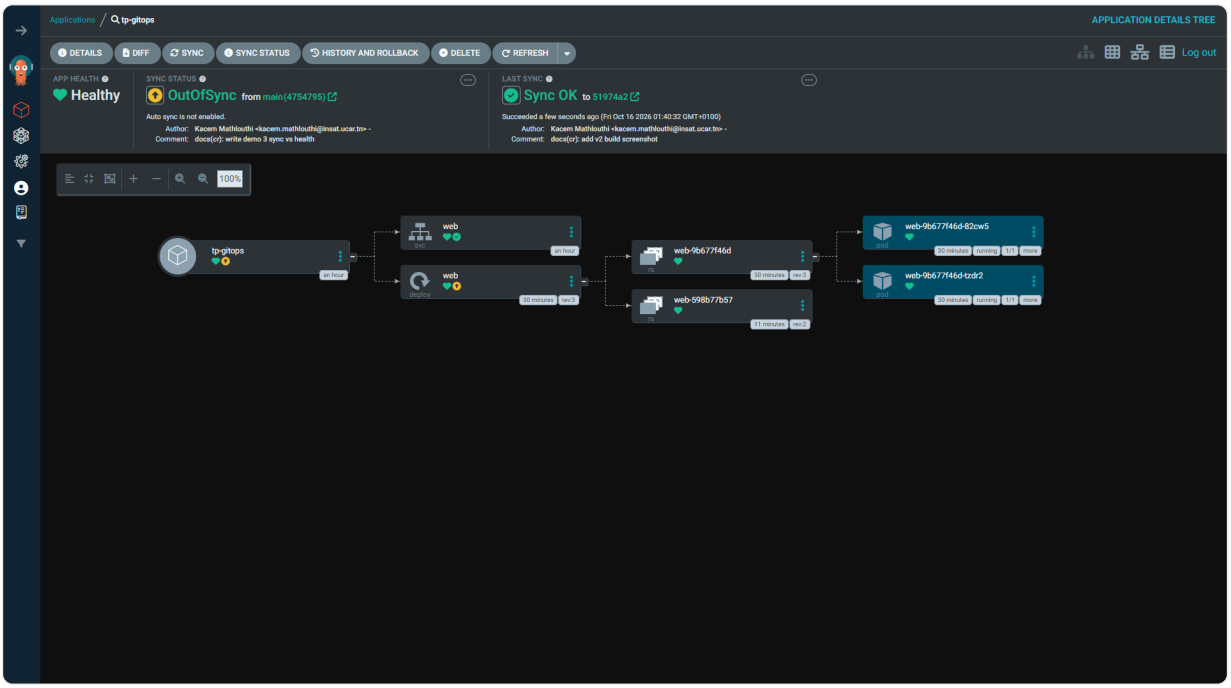


Fig. 30. Healthy, mais OutOfSync : Git référence toujours l'image cassée

- Quasi instantané, aucun nouveau pod : Kubernetes réutilise l'ancien ReplicaSet v2 (`rev:3`).
- OutOfSync est attendu : le cluster ne correspond plus à Git.
- Le rollback UI est une mesure d'urgence qui court-circuite la source de vérité.

Retour à Git

```
git revert 8a2a887   # newTag: doesnotexist → v2
git push
```

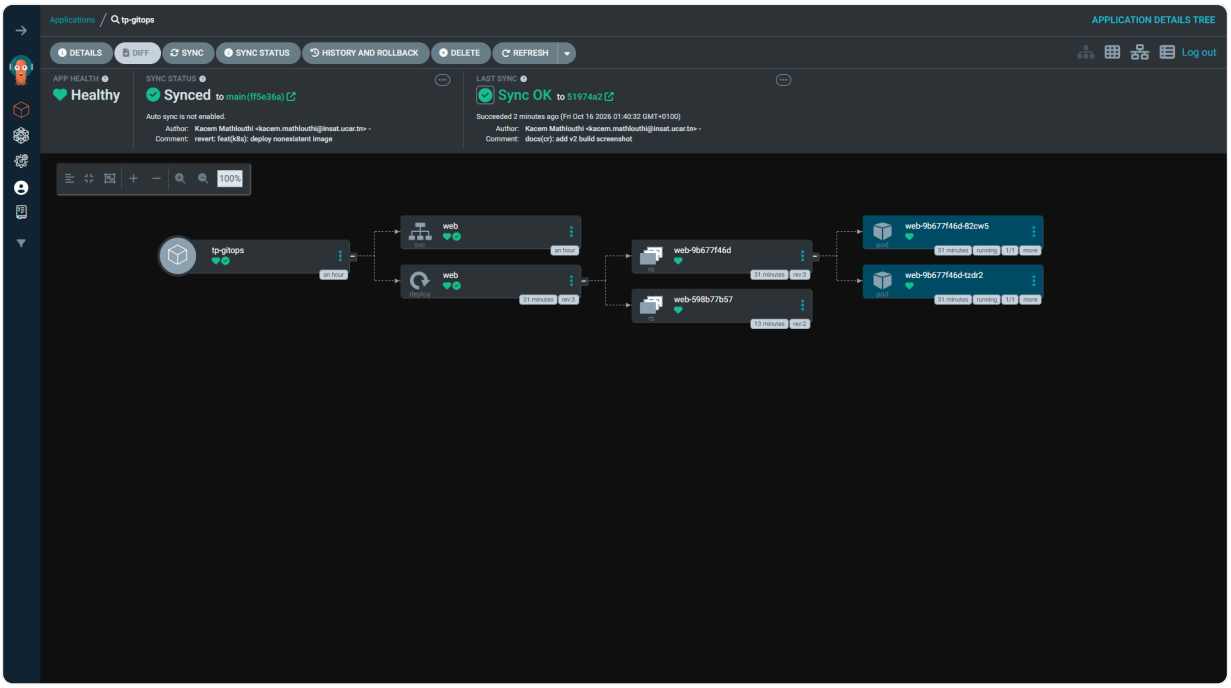


Fig. 31. Synced et Healthy sans aucune synchronisation : Git décrit de nouveau ce qui tourne

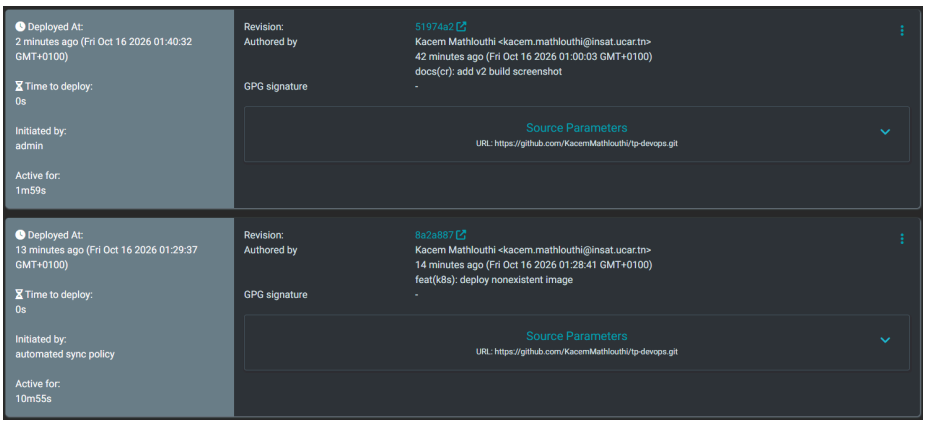


Fig. 32. Le rollback apparaît dans l'historique, initié par `admin`

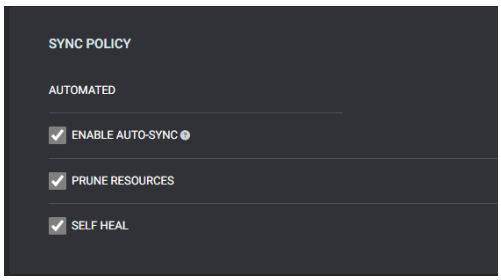


Fig. 33. Auto-sync réactivé

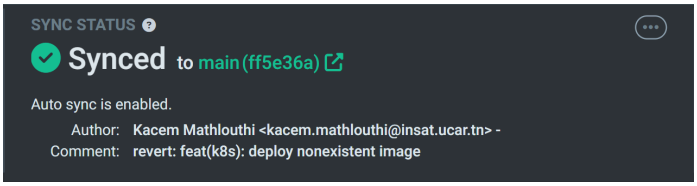


Fig. 34. État final : Synced, Auto sync is enabled

	Rollback ArgoCD (UI)	`git revert` + push
Délai	immédiat	polling ou Refresh
Traçabilité	historique ArgoCD	historique Git, relisible
Auto-sync	doit être désactivé	reste actif
Usage	urgence	correction pérenne

Résultat

- Retour à la dernière version saine en un clic, Healthy en quelques secondes.
- `git revert` puis réactivation de l'auto-sync : Git redevient la seule source de vérité.

Conclusion

Concepts démontrés

Single Source of Truth	tout l'état désiré est dans Git ; une seule commande `kubectl apply` (l'Application)
Pull-based CD	démo 1 : un commit d'une ligne déploie la v2, sans interruption
Drift et self-healing	démo 2 : Deployment supprimé recréé, scale manuel annulé, en quelques secondes
Sync et Health	démo 3 : image inexistante → Synced mais Degraded
Rollback et historique	démo 4 : rollback en un clic, puis `git revert`

Limites et pistes

- Polling de Git (3 min) : un webhook GitHub rendrait la détection immédiate.
- Le rollback UI impose de suspendre l'auto-sync et doit être suivi d'une correction dans Git.
- ArgoCD applique fidèlement une erreur : revue de code, vérification de l'image en CI ou déploiement progressif (Argo Rollouts) en amont.
- Base / overlay : ajouter un environnement `prod` = un overlay + une Application.

Dépôt

- GitHub : github.com/KacemMathlouthi/tp-devops